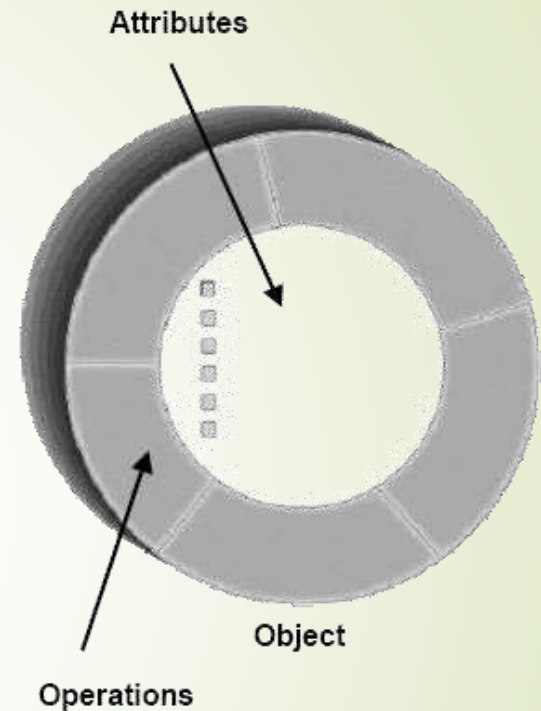

Nhắc lại kiến thức bài trước

Các kỹ thuật xây dựng lớp



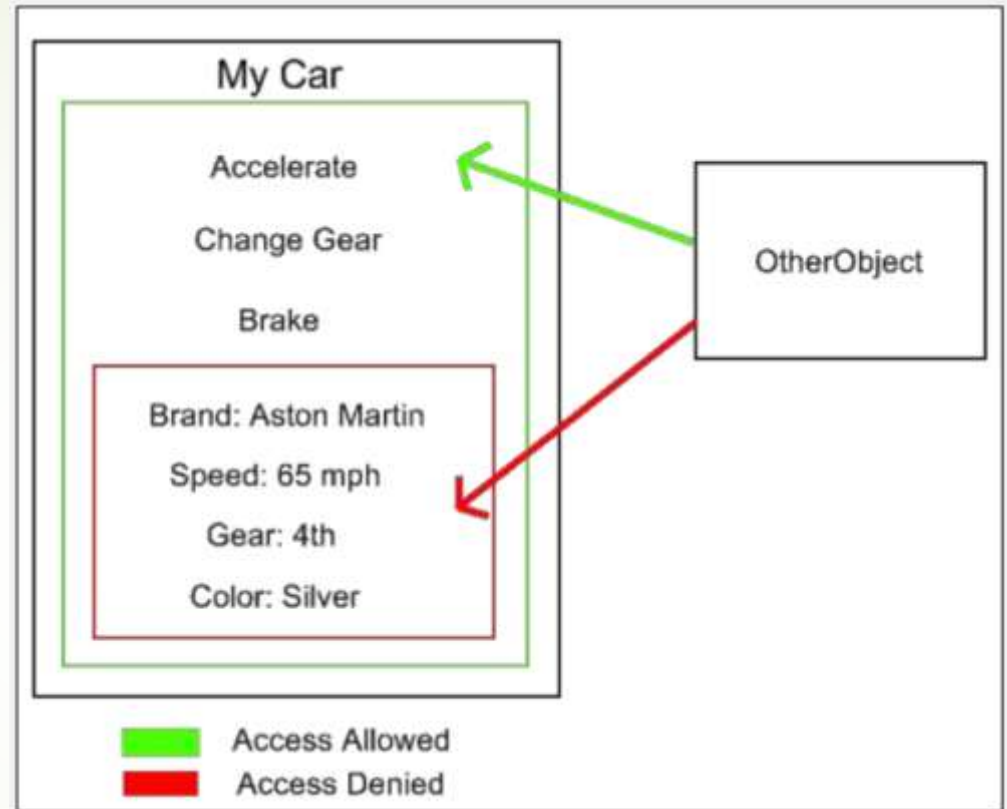
Đóng gói

- Đóng gói (Encapsulation)
 - Dữ liệu và phương thức được đóng gói trong một lớp
 - Dữ liệu được che giấu ở bên trong lớp và chỉ được truy cập và thay đổi ở các phương thức public



Che giấu dữ liệu

- Dữ liệu được che giấu ở bên trong lớp và chỉ được truy cập và thay đổi ở các phương thức bên ngoài
 - Tránh thay đổi trái phép hoặc làm sai lệch dữ liệu



Phương thức khởi tạo

- Ví dụ phương thức khởi tạo **không có tham số** (còn gọi là phương thức khởi tạo **mặc định**)

```
class BankAccount:  
  
    def __init__(self):  
        self.__owner = "  
        self.__balance = 0.0
```

Phương thức khởi tạo

- Ví dụ phương thức khởi tạo có tham số

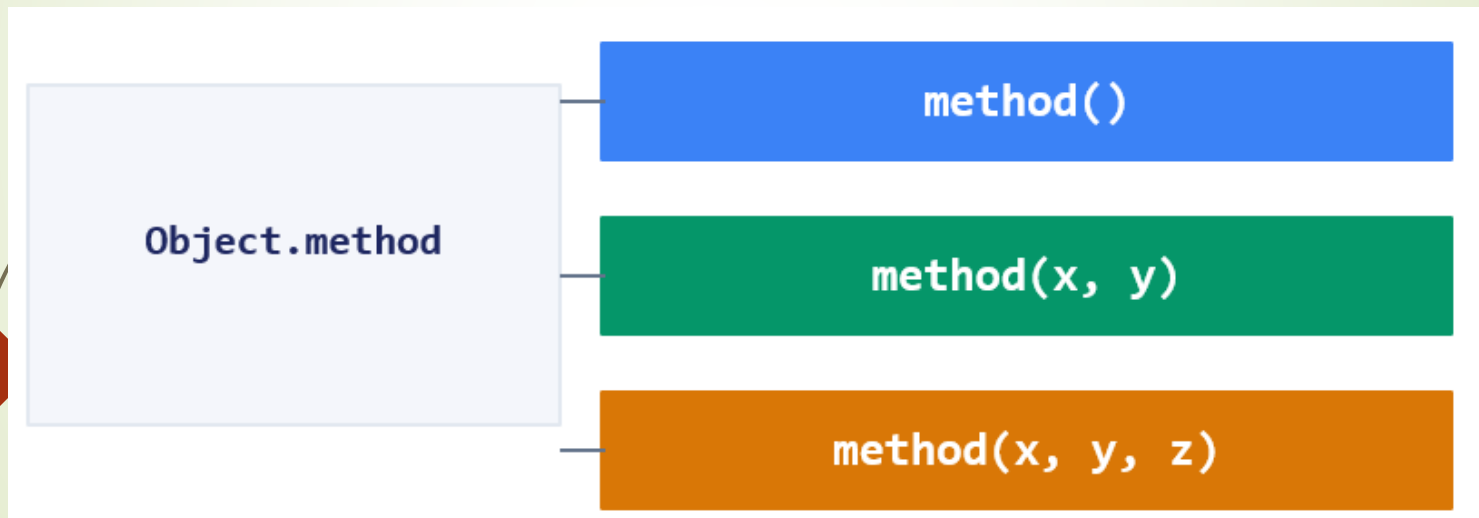
```
class BankAccount:  
  
    def __init__(self, name, balance):  
        self.__owner = name  
        self.__balance = balance
```

- Mục đích: giúp khởi tạo đối tượng dễ dàng hơn

```
account = BankAccount("Kiên", 2000000 )
```

Nạp chồng phương thức

- Hai hoặc nhiều phương thức có cùng tên nhưng khác số lượng tham số hoặc khác loại tham số hoặc cả hai. Các phương thức này được gọi là nạp chồng phương thức (**overloaded methods**) và điều này được gọi là nạp chồng (**overloading**).



Lưu ý:

Python KHÔNG hỗ trợ nạp chồng theo cách Java/C++. Khai báo hai hàm cùng tên → hàm sau ghi đè hàm trước, không báo lỗi.

Nạp chồng phương thức

Phương án 1

Khuyến nghị

***args**

Số lượng tham số biến thiên. Cần xử lý nhiều trường hợp đầu vào khác nhau.

Phương án 2

Đơn giản

Default params

Tình huống đơn giản, tối đa 2–3 trường hợp. Đọc và viết nhanh.

Phương án 3

Nâng cao

multipledispatch

Cần phân biệt theo kiểu dữ liệu tường minh. Khi môi trường cho phép thư viện ngoài.

Phương án	Cú pháp chính	Pythonic?	Phụ thuộc thư viện
*args	Kiểm tra len(args) hoặc isinstance()	Có	Không
Default params	Tham số =None, kiểm tra trong hàm	Có	Không
multipledispatch	Decorator @dispatch(type, type)	Tương đối	pip install

Trừu tượng hóa

“Bạn phải bắt đầu từ trải nghiệm của khách hàng, rồi mới làm ngược trở lại với công nghệ — chứ không phải ngược lại.”

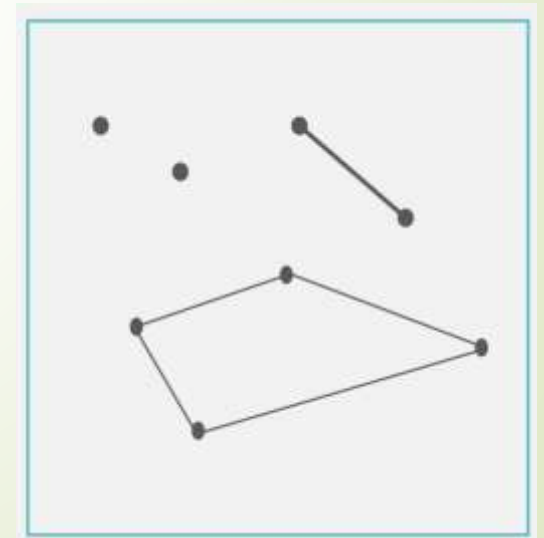
— Steve Jobs, 1997

[Start With the Customer, by Steve Jobs](#)



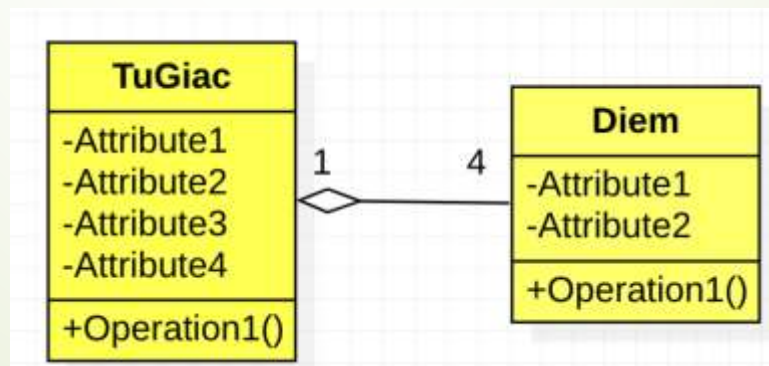
Mối quan hệ kết tập

- Lớp toàn thể chứa các đối tượng của lớp thành phần
 - Đối tượng lớp thành phần: Là một phần (**is-a-part of**) của lớp toàn thể
 - Quan hệ chứa/có ("**has-a**") hoặc là một phần ("is-a-part- of")
- Ví dụ
 - Tứ giác gồm 4 Điểm
 - Ô tô gồm 4 Bánh xe
 - Lớp học chứa 91 Sinh viên

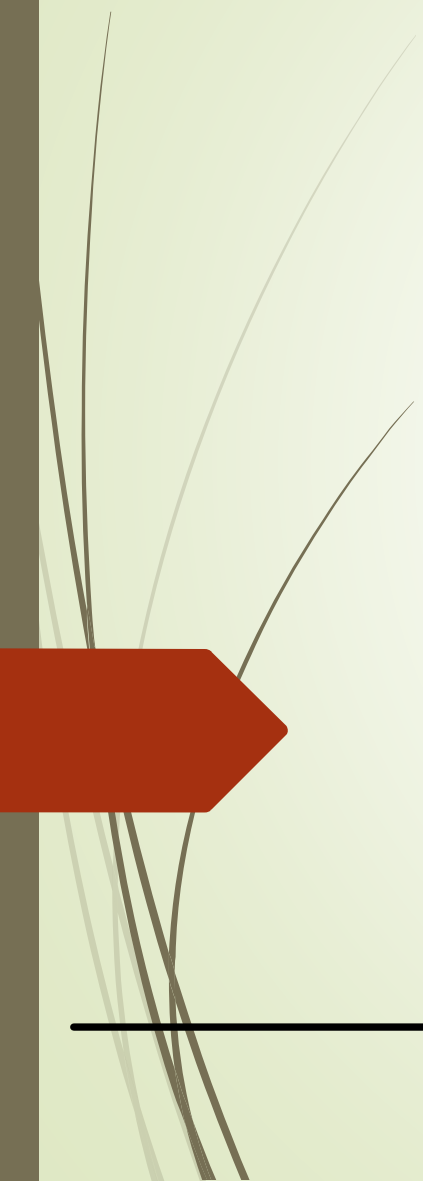


Biểu diễn kết tập bằng UML

- Sử dụng “hình thoi” tại đầu của lớp toàn thể
- Sử dụng bội số quan hệ (multiplicity) tại 2 đầu, có thể sử dụng:
 - Một số nguyên dương: 1, 2,...
 - Dải số: (0..1, 2..4)
 - *: Bất kỳ số nào
 - Không có: Mặc định là 1



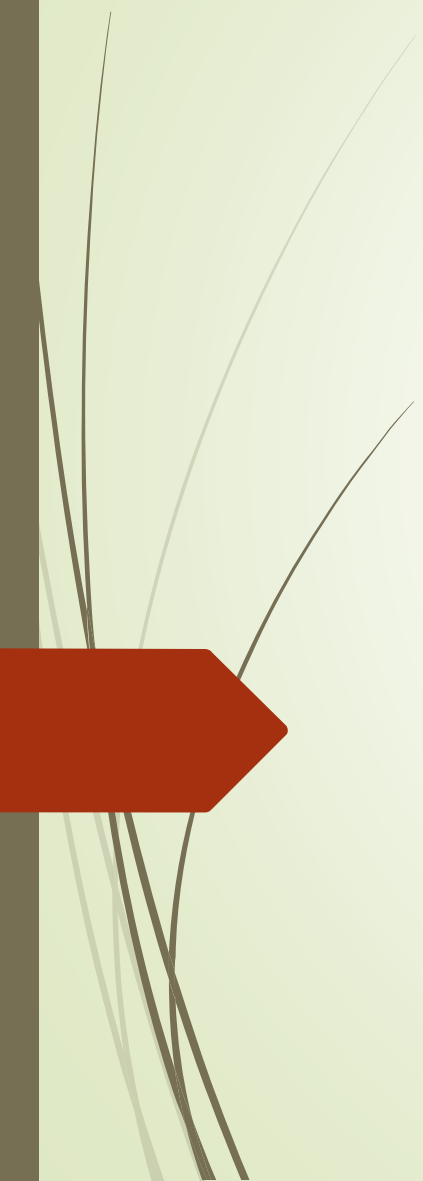
OOP05: Kế thừa



Nội dung

1. Khái niệm kế thừa
 2. Biểu diễn quan hệ kế thừa trong biểu đồ lớp
 3. Nguyên lý kế thừa
 4. Khởi tạo và hủy bỏ đối tượng lớp con
 5. Quan hệ thành phần
- 

1



Khái niệm kế thừa

Inheritance

Kế thừa

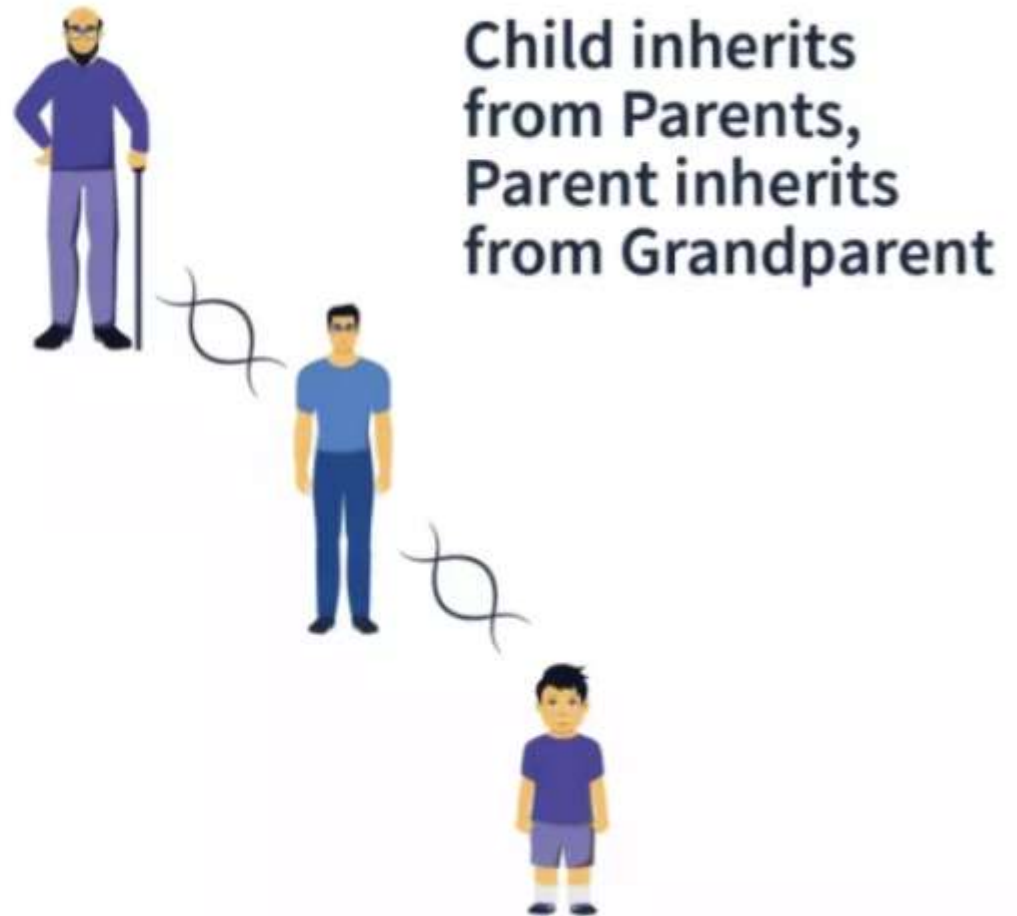
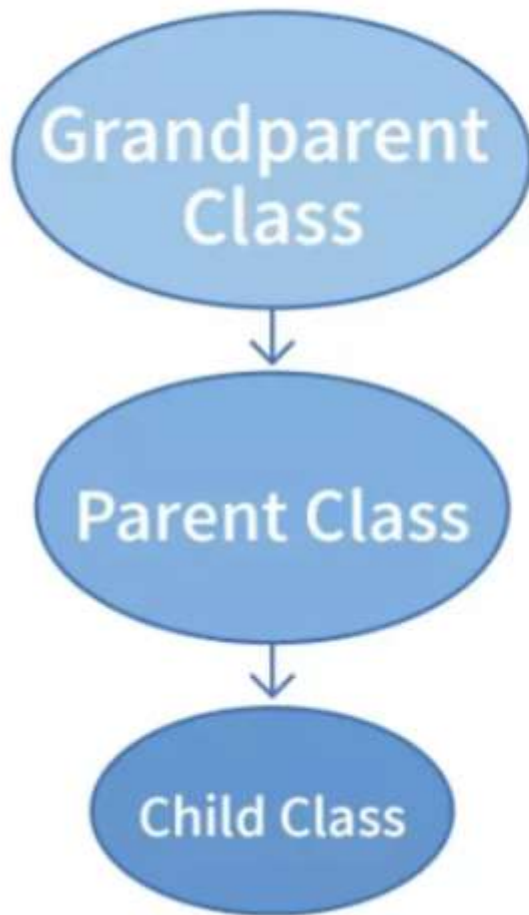
- Kế thừa?



“Xây dựng các lớp mới có sẵn các đặc tính của lớp cũ, đồng thời chia sẻ hay mở rộng các đặc tính sẵn có”

Kế thừa

- Kế thừa?

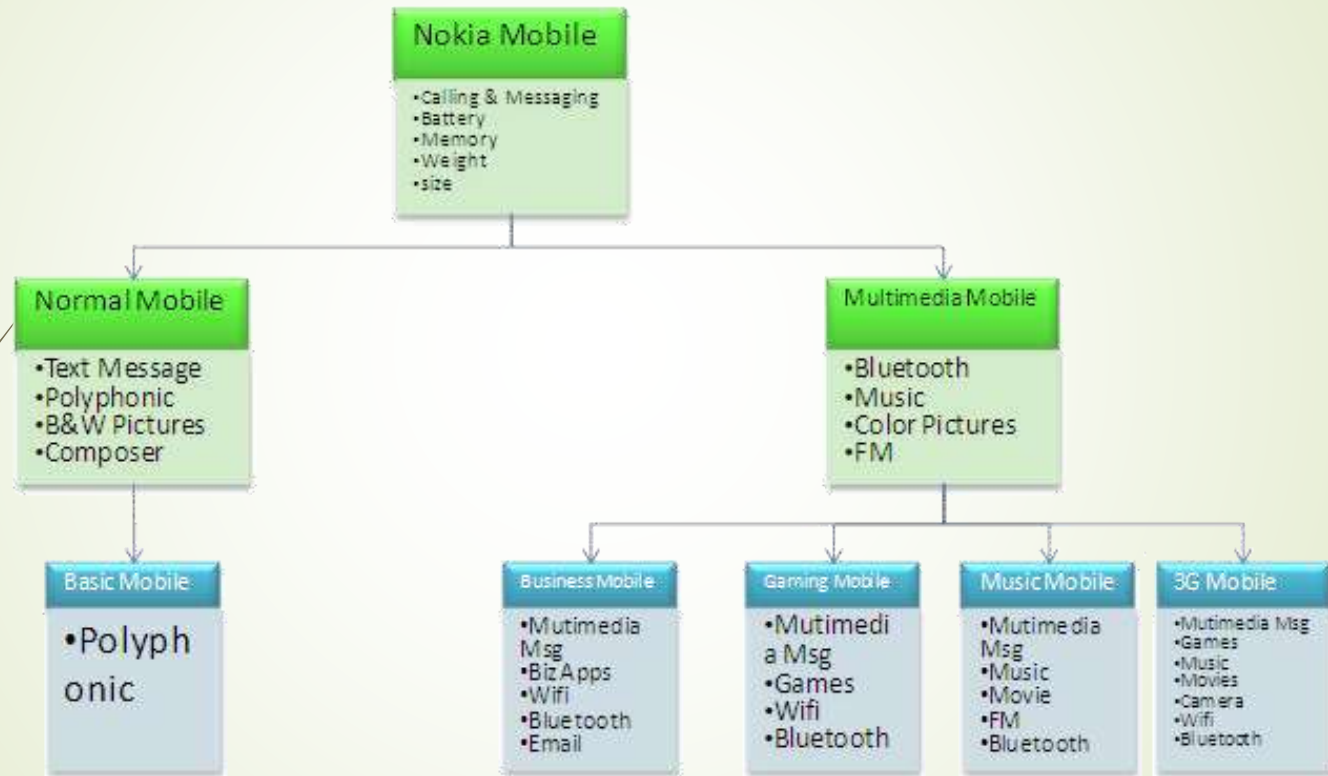


Bản chất kế thừa

- Phát triển lớp mới dựa trên các lớp đã có
- Ví dụ:
 - Lớp Người có các thuộc tính như tên, tuổi, chiều cao, cân nặng...; các phương thức như ăn, ngủ, chơi...
 - Lớp Sinh Viên thừa kế từ lớp Người, thừa kế được các thuộc tính tên, tuổi, chiều cao, cân nặng...; các phương thức ăn, ngủ, chơi...
 - Bổ sung thêm các thuộc tính như mã số sinh viên, số tín chỉ tích lũy..., các phương thức như tham dự lớp học, thi...

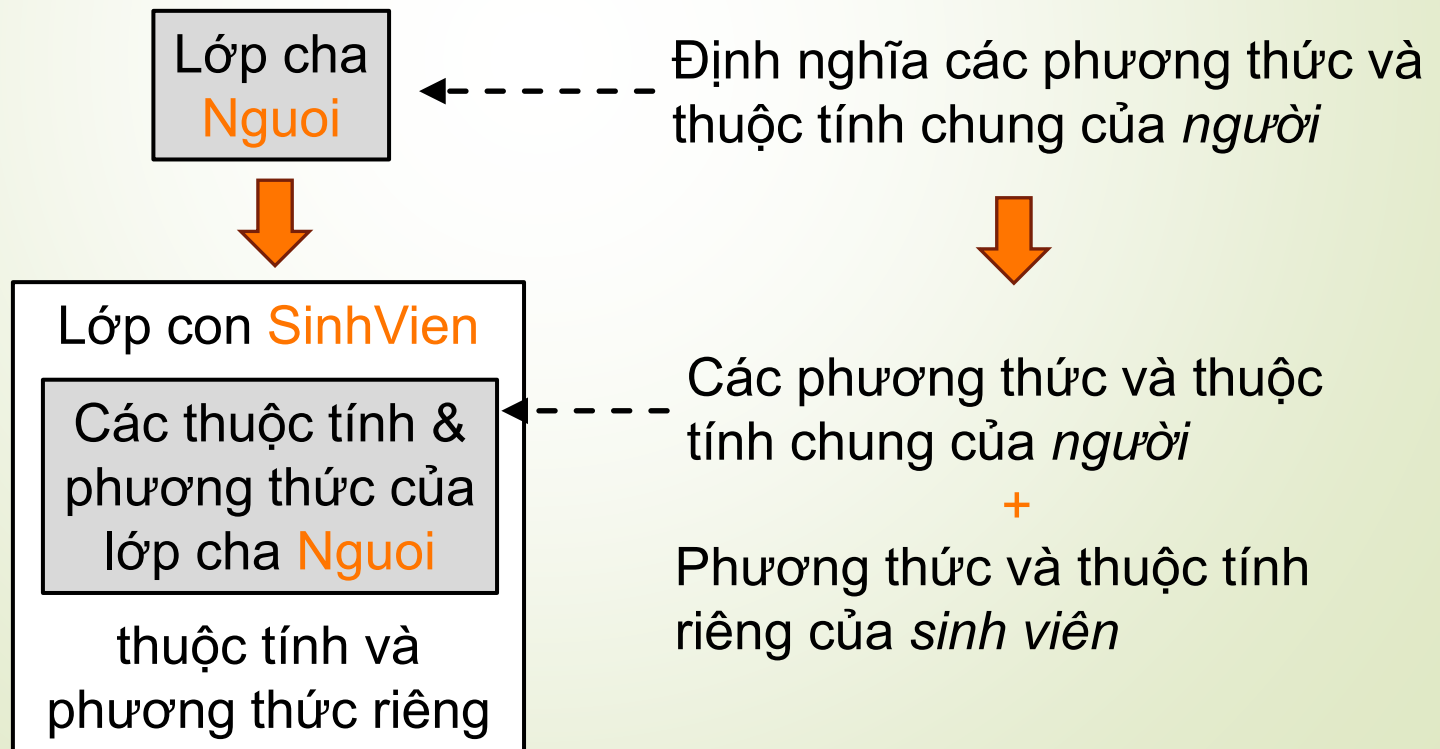
Bản chất kế thừa

- Chính là nguyên lý phân cấp trong trừu tượng hóa



Ví dụ

- Khái niệm
 - Lớp cũ: Lớp cha (parent, superclass), lớp cơ sở (base class)
 - Lớp mới: Lớp con (child, subclass), lớp dẫn xuất (derived class)
- Ví dụ: SinhVien thừa kế (dẫn xuất) từ lớp Nguoi



Mối quan hệ kế thừa

- Lớp con và lớp cha có tính tương đồng
 - Lớp SinhVien kế thừa từ lớp Nguoi.
 - Một sinh viên là một người

Lớp cha
Nguoi



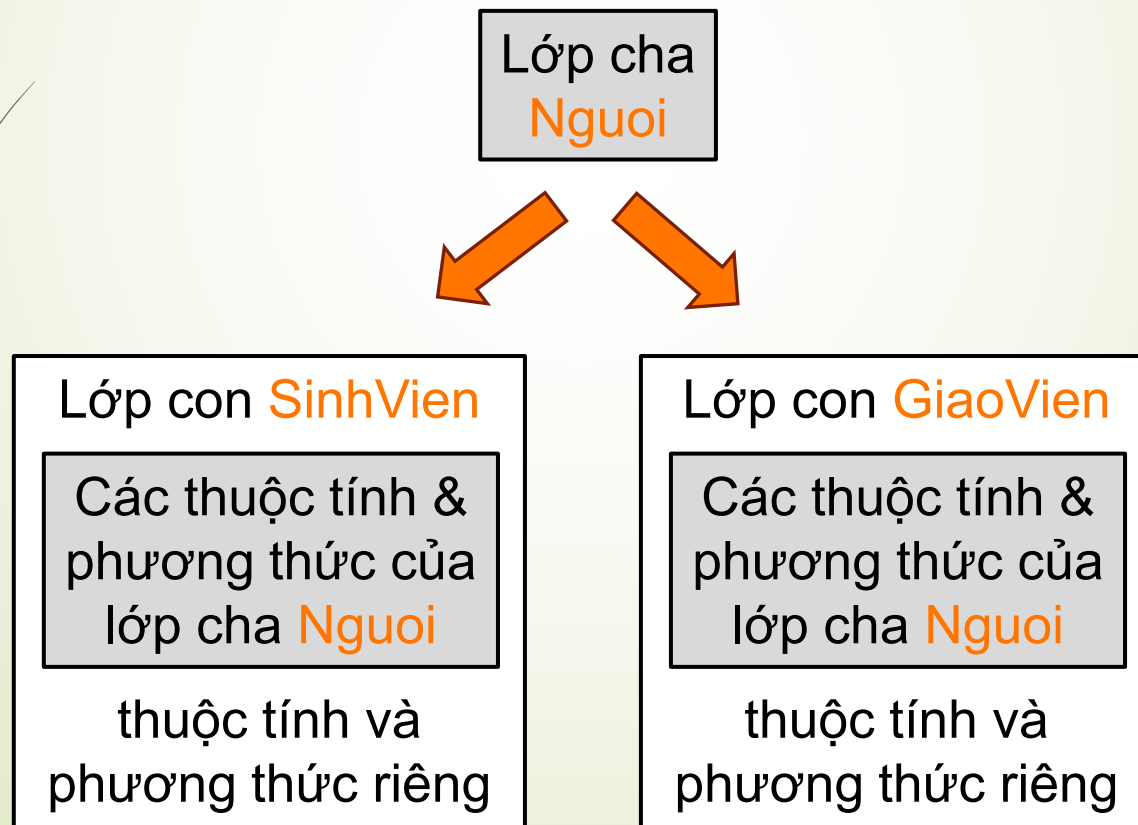
Lớp con SinhVien

Các thuộc tính &
phương thức của
lớp cha Nguoi

thuộc tính và
phương thức riêng

Mối quan hệ kế thừa

- Cả GiaoVien và SinhVien đều có quan hệ là (is-a) với lớp Ngươi
- Cả giáo viên và sinh viên đều có một số hành vi thông thường của con người



Mối quan hệ kế thừa

- Lớp con
 - Là *một loại* (is-a-kind-of) của lớp cha
 - Kế thừa các thành phần dữ liệu và các hành vi của lớp cha
 - Chi tiết hóa cho phù hợp với mục đích sử dụng mới
 - Extension: Thêm các thuộc tính/hành vi mới
 - Redefinition (Method Overriding): Chỉnh sửa/ghi đè các hành vi kế thừa từ lớp cha

Cú pháp (Python)

- Cú pháp (Python)
<lớp con> (<lớp cha>)

```
class Nguoi:  
    name = ""  
    age = 0  
  
class SinhVien(Nguoi):  
    student_id = 0
```

Lớp cha **Nguoi**
name, age



Lớp con **SinhVien**
name, age
studentId

Cú pháp (Java)

- Cú pháp (Java)

<lớp con> extends <lớp cha>

```
class Nguoi {  
    String name; int age;  
}  
  
class SinhVien extends Nguoi {  
    int studentId;  
}
```

Lớp cha **Nguoi**
name, age



Lớp con **SinhVien**
name, age
studentId

Cú pháp (C++, C#)

- Cú pháp (C++)
<lớp con> : <lớp cha>

```
#include <string>
class Nguoi {
    std::string name; int age;
};

class SinhVien : Nguoi {
    int studentId;
};
```

Lớp cha **Nguoi**
name, age



Lớp con **SinhVien**
name, age
studentId

Bản chất kế thừa

- Là một kỹ thuật *tái sử dụng mã nguồn* (*code reuse*)
 - Tái sử dụng mã nguồn thông qua *lớp*
- Ví dụ: Lớp Sinh Viên tái sử dụng được các thuộc tính như tên, tuổi... và các phương thức của lớp Người.

Kế thừa và kết tập

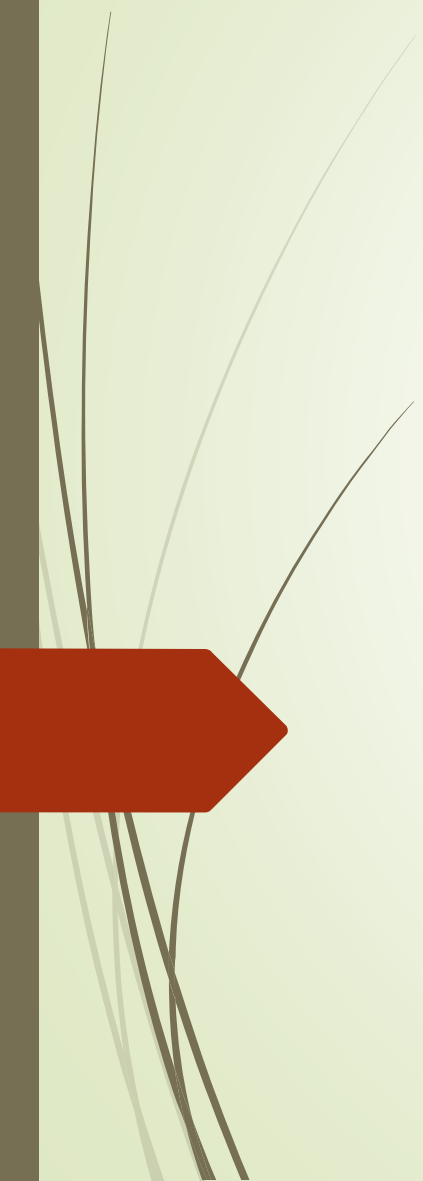
Kế thừa

- Tái sử dụng thông qua *lớp*
- Tạo lớp mới bằng cách phát triển lớp đã có sẵn
- Quan hệ “*là một loại*” (is a kind of)

Kết tập

- Tái sử dụng thông qua *đối tượng*
- Tạo ra tham chiếu đến các đối tượng của các lớp có sẵn trong lớp mới
- Quan hệ “*là một phần*” (is a part of)

2

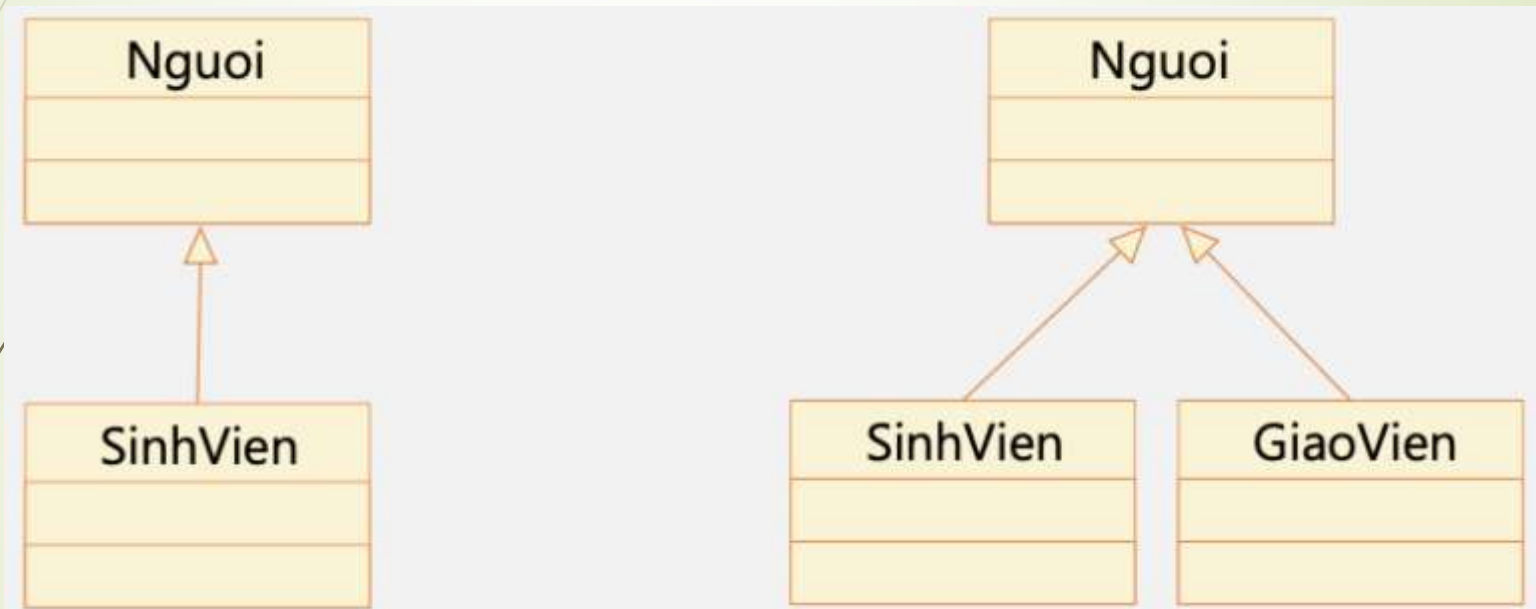


Biểu diễn quan hệ kế thừa trong sơ đồ lớp

Inheritance representation in class diagram

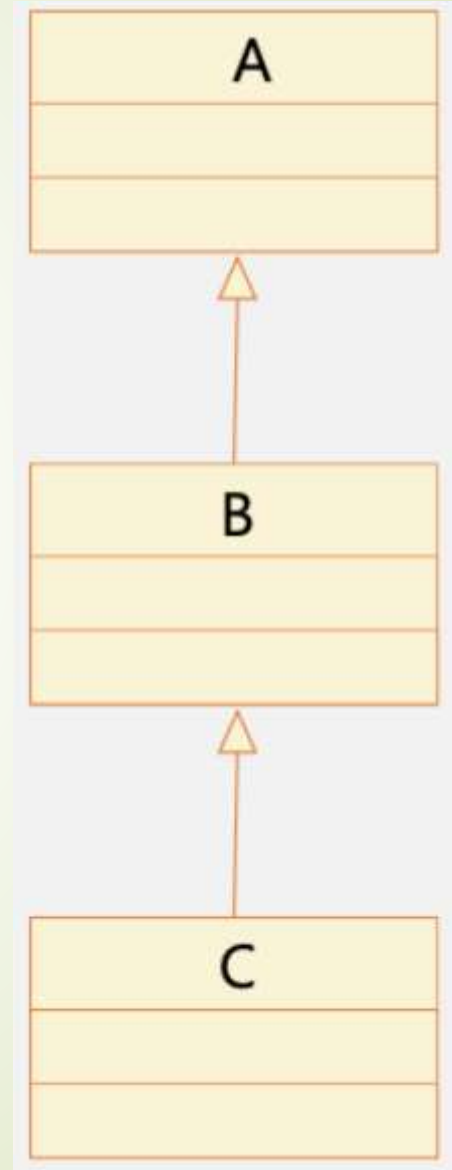
Cây phân cấp kế thừa

- Dùng mũi tên với tam giác rỗng ở đầu



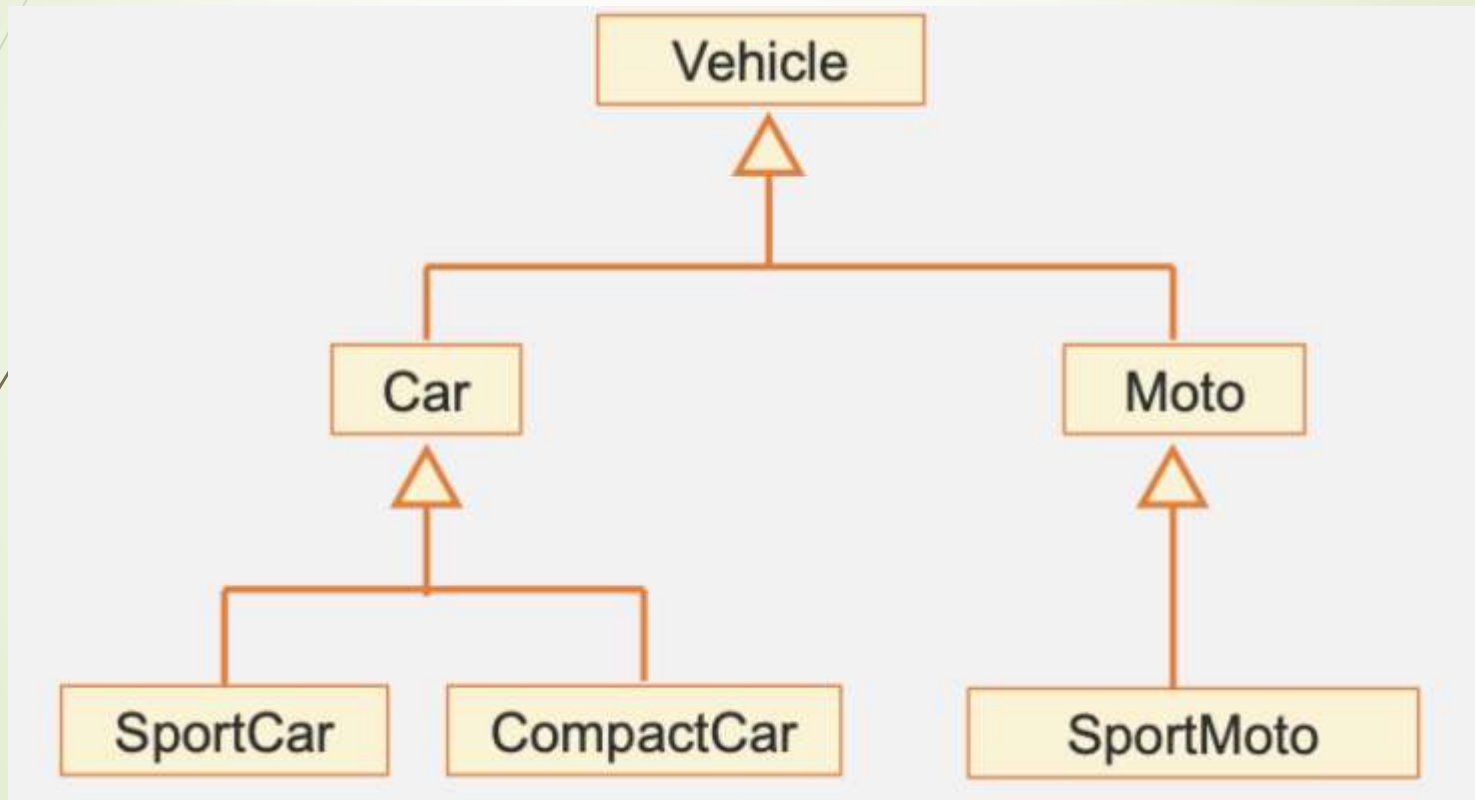
Cây phân cấp kế thừa

- Cấu trúc phân cấp hình cây, biểu diễn mối quan hệ kế thừa giữa các lớp.
- Dẫn xuất trực tiếp
 - B dẫn xuất trực tiếp từ A
- Dẫn xuất gián tiếp
 - C dẫn xuất gián tiếp từ A



Cây phân cấp kế thừa

- Các lớp con có cùng lớp cha gọi là các lớp anh chị em (siblings)



Lớp Objects

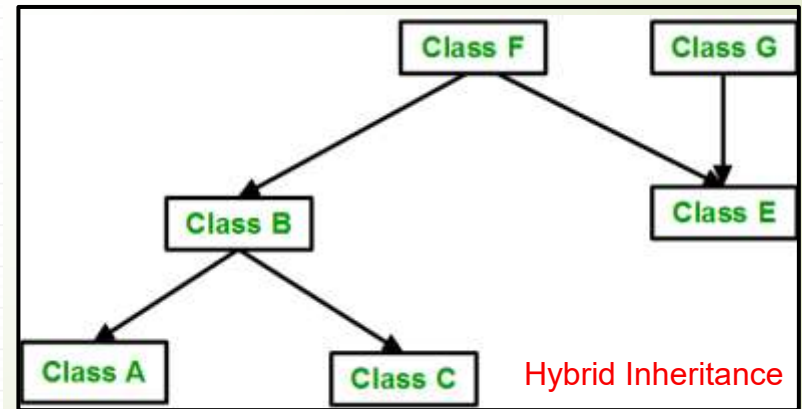
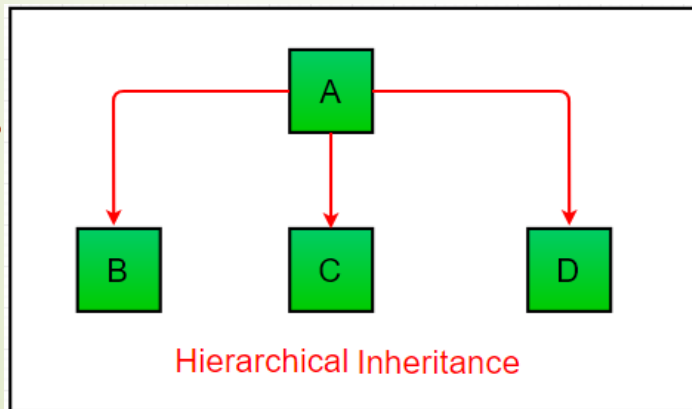
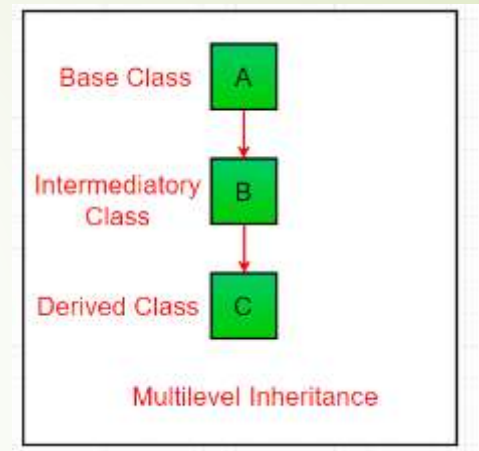
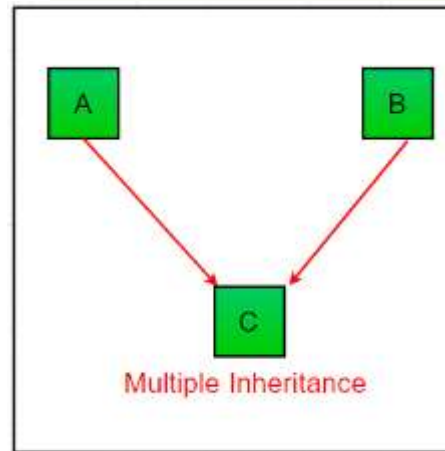
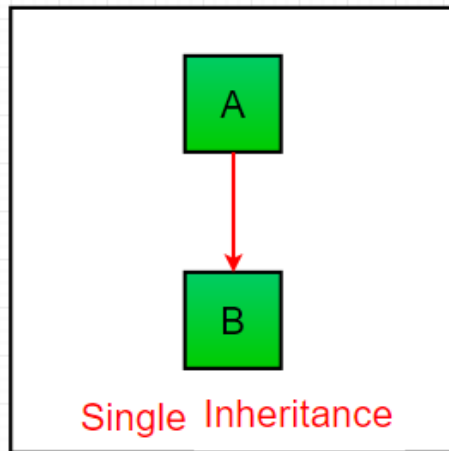
- Lớp **Object** là lớp gốc trên cùng của tất cả các cây phân cấp kế thừa
 - Nếu một lớp không được định nghĩa là lớp con của một lớp khác thì mặc định nó là lớp con trực tiếp của lớp **Object**
 - Chứa một số ít phương thức hữu ích kế thừa lại cho tất cả các lớp, thường được đặt tên với dấu gạch dưới kép (double-underscore), ví dụ: `__init__`, `__str__`, `__repr__`, `__eq__`, `__hash__`, ...

```
# Mọi lớp đều kế thừa từ object
class Ngươi:
    pass

print(Ngươi.__bases__)    # (<class
'object'>,)
print(isinstance(Ngươi(), object))  # True
```

Các loại kế thừa

- Các loại kế thừa trong Python:



3

Nguyên lý kế thừa

Lớp con kế thừa được những gì từ lớp cha

Nguyên lý kế thừa

- Lớp con có thể thừa kế được gì từ lớp cha?
 - Kế thừa được các thành viên được khai báo là **public** và **protected** của lớp cha.
 - Không kế thừa được các thành viên **private**.
- Thành viên **protected** trong lớp cha được truy cập trong:
 - Các thành viên lớp cha
 - Các thành viên lớp con

Nguyên lý kế thừa

Chú ý:

- Trong Python, **access modifier** là quy ước (convention), không phải cơ chế cứng như Java/C++:
 - public (không tiền tố): Kế thừa và truy cập tự do
 - `_protected` (1 gạch dưới): Kế thừa bình thường, quy ước chỉ dùng nội bộ.
 - `__private` (2 gạch dưới): Kế thừa nhưng bị name mangling - tên bị đổi thành: `_TenLop__tenThuocTinh`, nên lớp con không truy cập trực tiếp.

Ví dụ

=== PUBLIC ===

Public access

```
class Person:
    def __init__(self):
        self.name = 'A'      # public
        self.age = 18

class Student(Person):
    def __init__(self):
        super().__init__()
        self.student_id = 20231234

s = Student()
print(s.name)      # 'A'   ✓ Kế thừa OK
print(s.age)       # 18    ✓ Kế thừa OK
print(s.student_id) # 20231234
```

=== PROTECTED (quy ước) ===

Protected access

```
class Person:
    def __init__(self):
        self._name = 'A'    # protected (quy ước)
        self._age = 18

class Student(Person):
    def __init__(self):
        super().__init__()
        self.student_id = 20231234

    def info(self):
        # Lớp con truy cập _protected bình thường
        print(f"Tên: {self._name}, Tuổi: {self._age}")

s = Student()
s.info()
# Tên: A, Tuổi: 18   ✓ Lớp con truy cập OK
print(s._name)
# 'A'   ⚠ Vẫn truy cập được nhưng KHÔNG NÊN
```

Ví dụ

Tại sao lại ko nên sử dụng cách gọi **s._name** trong protected. Hãy xem xét ví dụ này.

```
class BankAccount:
    def __init__(self):
        self._balance = 1000

    def withdraw(self, amount):
        if amount > self._balance:
            print("Không đủ tiền!")
            return
        self._balance -= amount
```

Nếu bên ngoài gọi **acc._balance = -999999** — nó **bỏ qua** hoàn toàn logic kiểm tra trong **withdraw()**, dữ liệu sẽ sai. Method **withdraw()** là **cửa chính** có bảo vệ, còn **_balance** là **cửa sau** không ai canh.

Ví dụ

Private access

```
# === PRIVATE (name mangling) ===
class Person:
    def __init__(self):
        self.__name = 'A'      # private → bị đổi tên thành _Person__name
        self.__age = 18

class Student(Person):
    def __init__(self):
        super().__init__()
        self.student_id = 20231234

    def info(self):
        # print(self.__name)      ✗ AttributeError!
        # Phải dùng name mangling:
        print(self._Person__name) # ⚠ Hoạt động nhưng KHÔNG NÊN

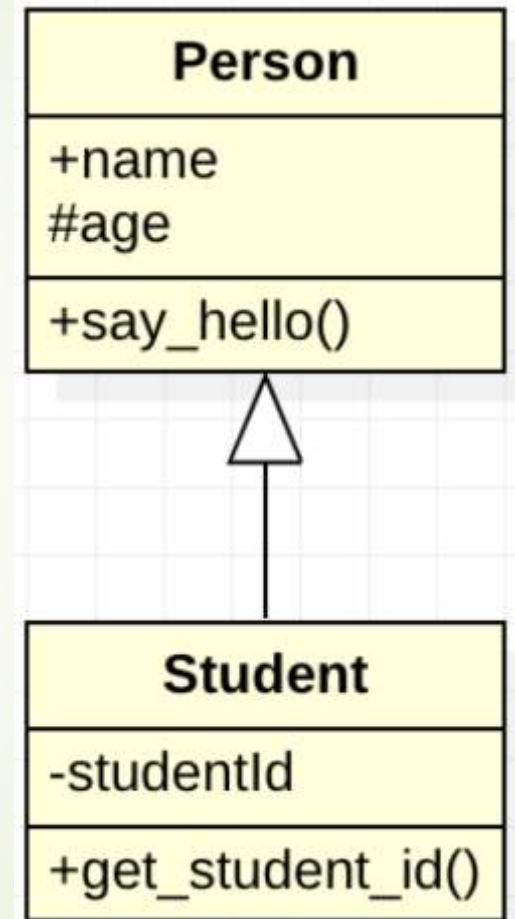
s = Student()
# print(s.__name)                ✗ AttributeError: 'Student' has no attribute '__name'
print(s._Person__name)          # 'A' ← name mangling, thuộc tính vẫn tồn tại!
```

Ví dụ

- Viết mã định nghĩa lớp Student kế thừa từ lớp Person với các thuộc tính và phương thức như trong sơ đồ lớp sau:
- Ký hiệu phạm vi truy cập:



⚭ ⁺	public
⚭ [#]	protected
⚭ ⁻	private
⚭ [~]	package



Ví dụ

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self._age = age

    def say_hello(self):
        print(f"Hello, my name is {self.name} and I am {self._age} years old.")

class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age)
        self.__student_id = student_id

    def say_hello(self):
        super().say_hello()
        print(f"My student ID is {self.__student_id}.")

    def get_student_id(self):
        return self.__student_id
```

```
person = Person("John", 25)
person.say_hello() # prints "Hello, my name is John and I am 25 years old."
```

```
student = Student("Jane", 21, "1234")
student.say_hello()
# prints "Hello, my name is Jane and I am 21 years old."
# prints "My student ID is 1234."
```

```
# Truy cập các thuộc tính công khai của lớp Person
print(student.name) # prints "Jane"
print(student._age) # prints 21
```

```
# Dòng dưới sẽ gây AttributeError vì __student_id là private
# Python đổi tên thành _Student__student_id (name mangling)
```

```
print(student.__student_id) # ❌ AttributeError!
# Thay vào đó, dùng getter:
```

```
print(student.get_student_id()) # ✅ "1234"
```

4

Khởi tạo và hủy bỏ đối tượng

Thứ tự khởi tạo, gọi phương thức của lớp cha

Khởi tạo và hủy bỏ đối tượng

- Khởi tạo và hủy bỏ đối tượng trong kế thừa
- Khởi tạo đối tượng:
 - Lớp cha được khởi tạo trước lớp con.
 - Các phương thức khởi tạo của lớp con luôn gọi phương thức khởi tạo của lớp cha ở câu lệnh đầu tiên
 - Tự động gọi (ngầm định - implicit): Khi lớp cha CÓ phương thức khởi tạo mặc định (hoặc ngầm định)
 - Gọi trực tiếp (tường minh - explicit)
- Hủy bỏ đối tượng:
 - Ngược lại so với khởi tạo đối tượng

Sử dụng từ khóa Super

Gọi phương thức của lớp cha

- Tái sử dụng các đoạn mã của lớp cha trong lớp con
- Gọi phương thức khởi tạo (sử dụng từ khóa **super**):
 - Bắt buộc nếu lớp cha không có phương thức khởi tạo mặc định
 - Phải được khai báo tại dòng lệnh đầu tiên trong phương thức khởi tạo của lớp con
- Gọi các phương thức của lớp cha

Python:

```
super().__init__(name, age)
super().say_hello()
```

Java:

```
super(name, age);           // trong constructor
super.sayHello();           // gọi method lớp cha
```

C++:

```
Person(name, age)           // trong danh sách khởi tạo
Person::sayHello();          // gọi method lớp cha
```

Phương thức Hủy của lớp cha

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print("Person constructor called")

    def say_hello(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years old.")

    def __del__(self):
        print("Person destructor called")
```

⚠ **Lưu ý:** Ví dụ `__del__` chỉ mang tính minh họa khái niệm destructor. Trong thực tế Python, bạn hiếm khi cần viết `__del__` vì Python có garbage collector tự động. Nếu cần dọn tài nguyên (file, kết nối DB...), hãy dùng **context manager** (with) hoặc phương thức `close()` thay thế.

Gọi phương thức Hủy của lớp cha

```
class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age) # Gọi constructor lớp cha TRƯỚC
        self.student_id = student_id
        print("Student constructor called")

    def say_hello(self):
        super().say_hello() # Gọi method lớp cha
        print(f"My student ID is {self.student_id}.")

    def __del__(self):
        print("Student destructor called")
        # Lưu ý: Không cần gọi super().__del__() trong Python thông thường
        # Vì sẽ tự xử lý. Chỉ minh họa để hiểu thứ tự này.
```

```
# Thứ tự khởi tạo: Person → Student
student = Student("Jane", 21, "1234")
# Output:
# Person constructor called
# Student constructor called
```

```
student.say_hello()
# Output:
# Hello, my name is Jane and I am 21 years old.
# My student ID is 1234.
```

```
del student
# Output:
# Student destructor called
```

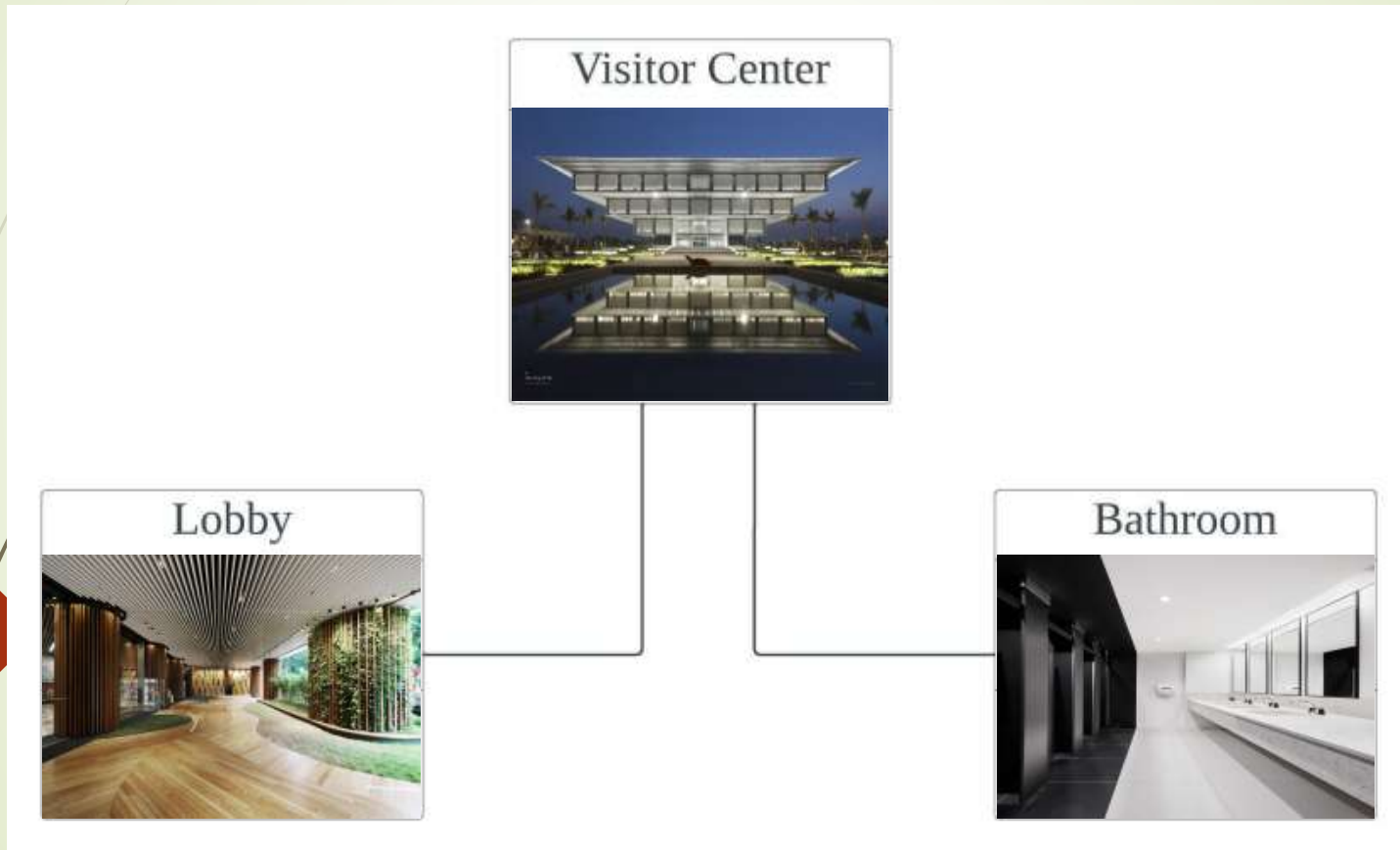
5

Quan hệ thành phần

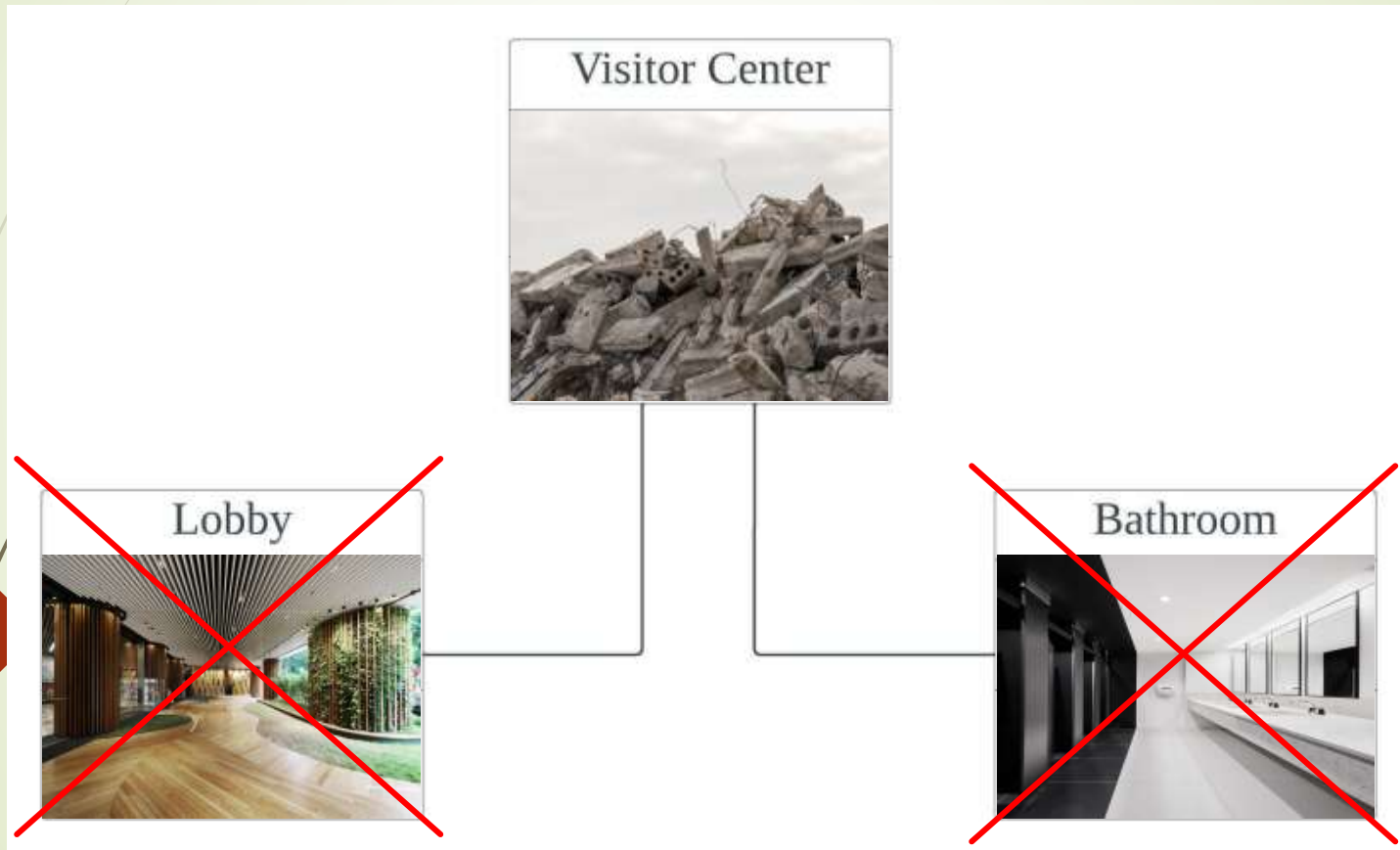
Composition



Quan hệ thành phần



Quan hệ thành phần



Quan hệ thành phần

```
# === COMPOSITION: Đối tượng thành phần được TẠO BÊN TRONG lớp toàn thể ===  
# Khi lớp toàn thể bị hủy → thành phần cũng bị hủy
```

```
class Lobby:
```

```
    def __init__(self, area):  
        self.area = area  
        print(f" Lobby ({self.area}m2) được tạo")
```

```
    def __del__(self):  
        print(f" Lobby ({self.area}m2) bị hủy")
```

```
class Bathroom:
```

```
    def __init__(self, floor):  
        self.floor = floor  
        print(f" Bathroom (tầng {self.floor}) được tạo")
```

```
    def __del__(self):  
        print(f" Bathroom (tầng {self.floor}) bị hủy")
```

```
class VisitorCenter:
```

```
    def __init__(self, name):  
        self.name = name  
        print(f"Xây dựng {name}...")  
        # COMPOSITION: Tạo đối tượng thành phần BÊN TRONG constructor  
        # Lobby, Bathroom KHÔNG tồn tại độc lập ngoài VisitorCenter  
        self.lobby = Lobby(200)          # tạo mới, không nhận từ bên ngoài  
        self.bathrooms = [Bathroom(i) for i in range(1, 4)]
```

```
    def __del__(self):  
        print(f"Phá hủy {self.name} → mọi thành phần bị hủy theo!")
```

```
# Demo
```

```
vc = VisitorCenter("Bảo tàng Hà Nội")
```

```
del vc # Hủy VisitorCenter → Lobby + 3 Bathroom cũng bị hủy
```

Quan hệ kết tập / Quan hệ thành phần

```
# === SO SÁNH: AGGREGATION vs COMPOSITION ===
```

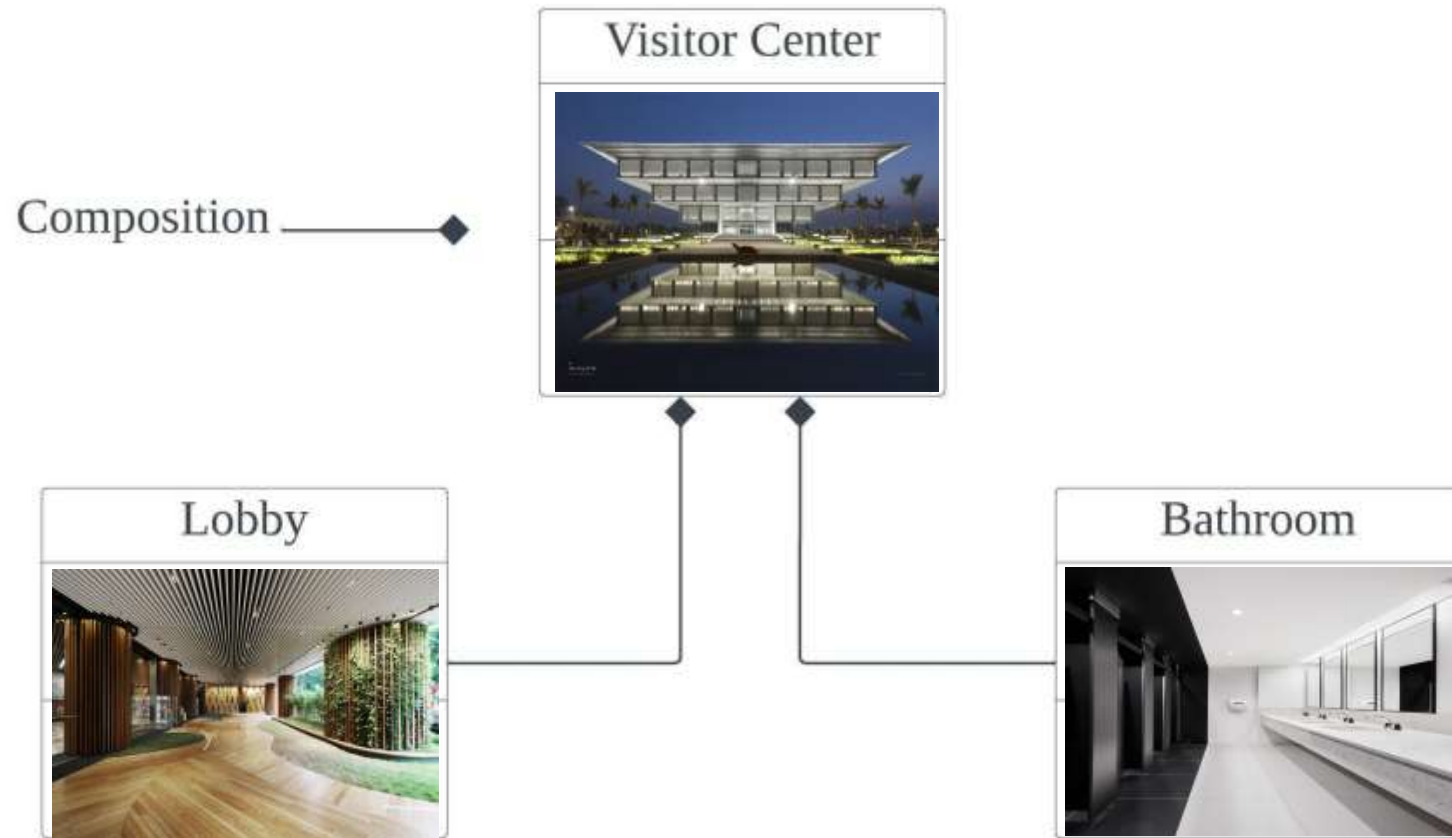
```
# AGGREGATION: Đối tượng thành phần được TRUYỀN TỪ BÊN NGOÀI  
# → Thành phần có thể tồn tại độc lập
```

```
class LopHoc_Aggregation:  
    def __init__(self, ten_lop, danh_sach_sv):  
        self.ten_lop = ten_lop  
        self.sinh_vien = danh_sach_sv # nhận từ bên ngoài
```

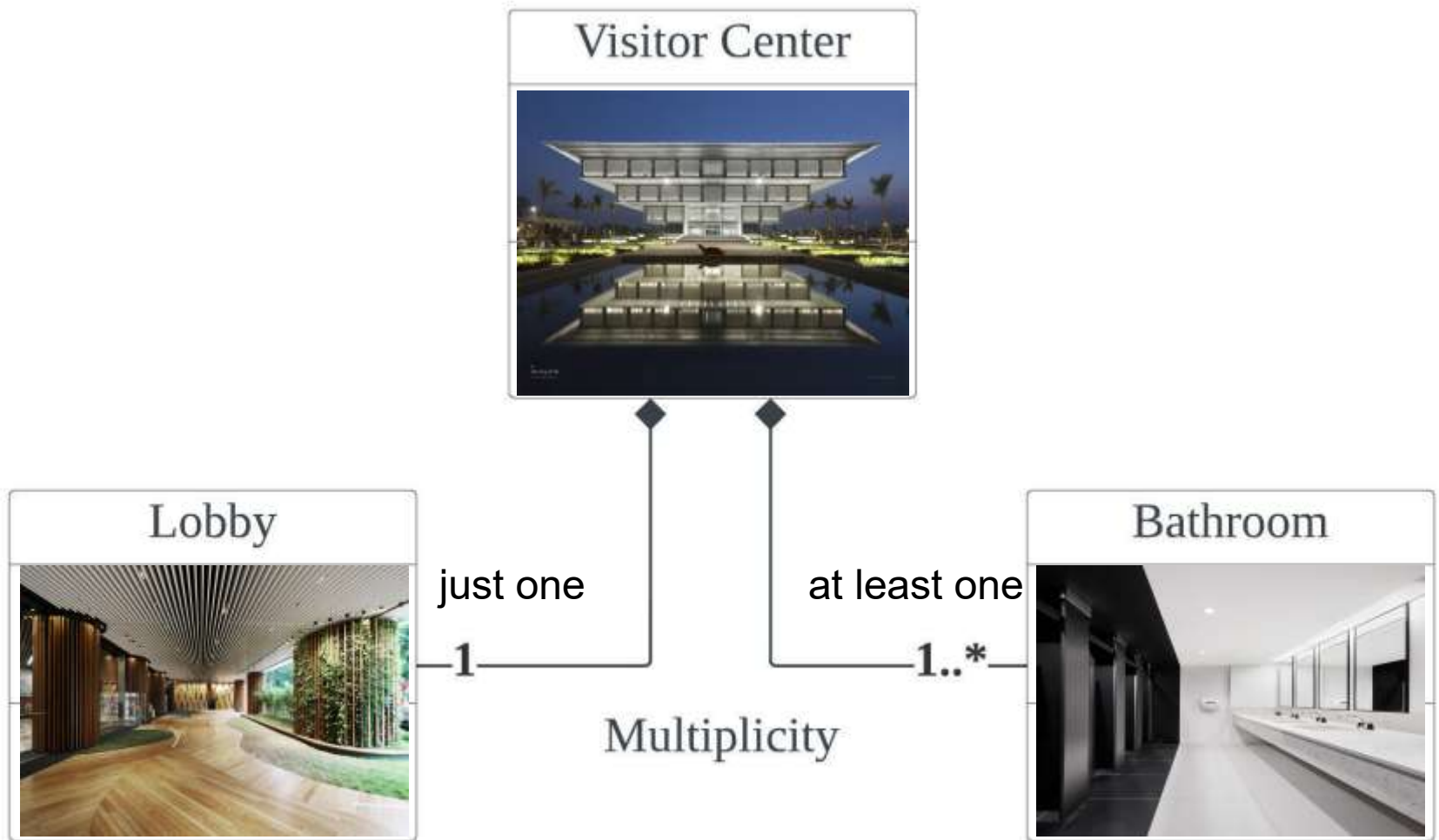
```
# COMPOSITION: Đối tượng thành phần được TẠO BÊN TRONG  
# → Thành phần không tồn tại độc lập
```

```
class LopHoc_Composition:  
    def __init__(self, ten_lop, so_ban):  
        self.ten_lop = ten_lop  
        self.ban_hoc = [BanHoc(i) for i in range(so_ban)] #  
tạo mới bên trong
```

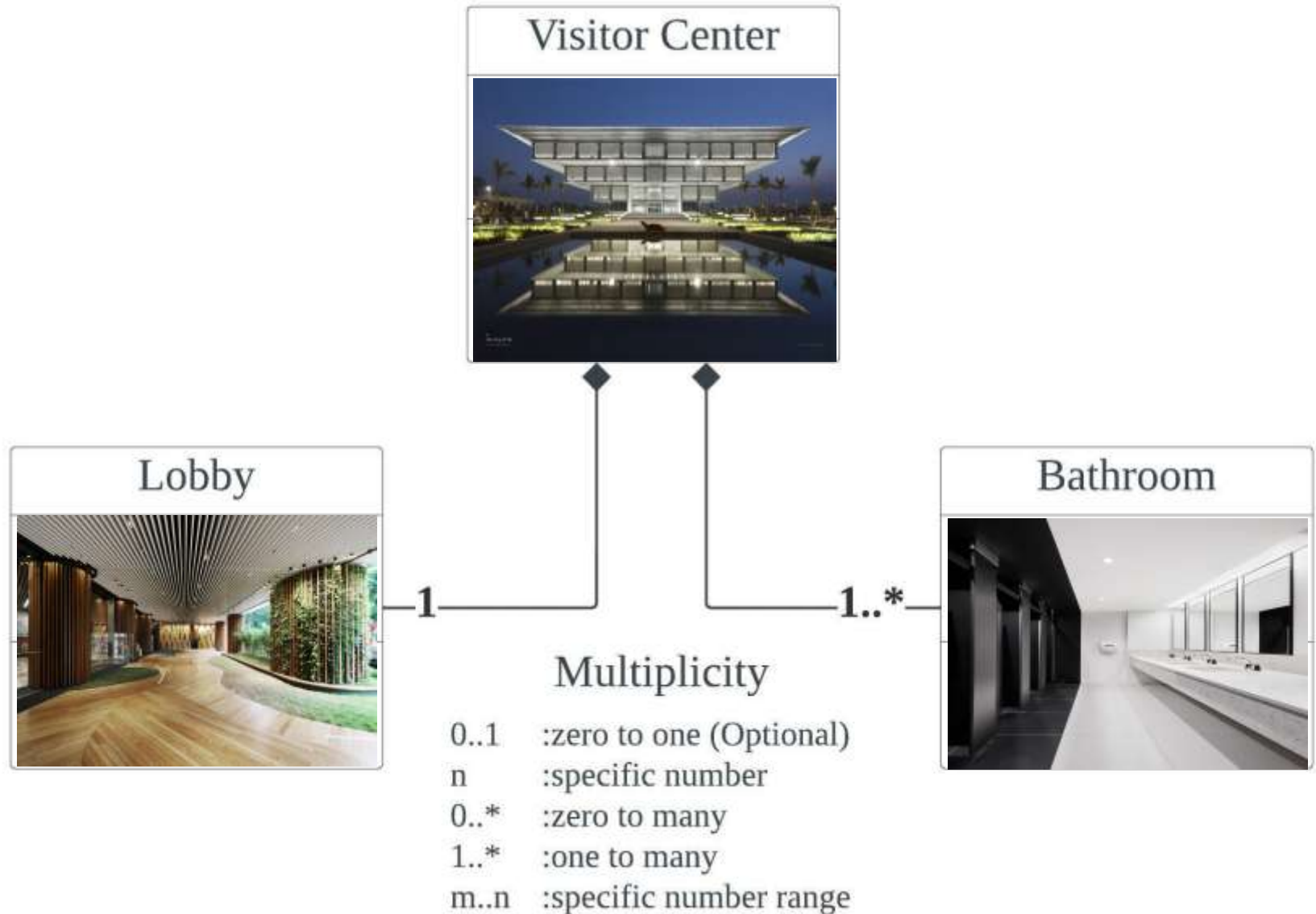
Quan hệ thành phần



Quan hệ thành phần



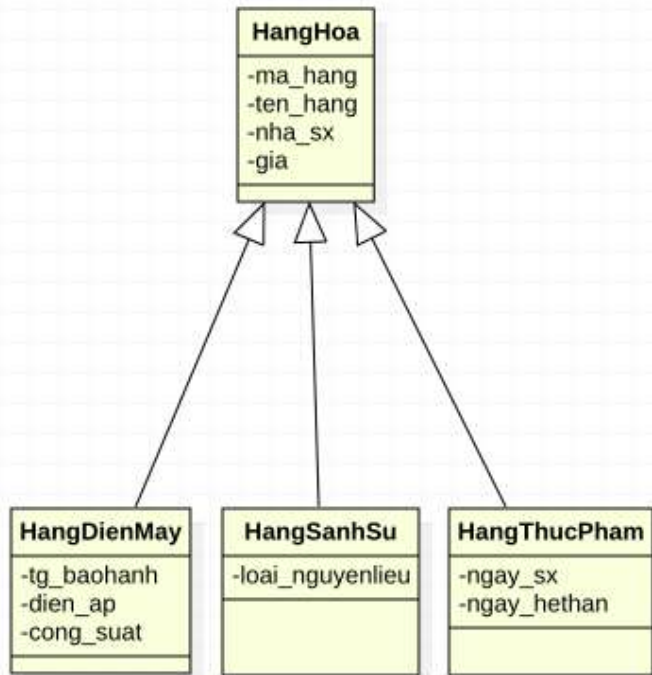
Quan hệ thành phần



Quy tắc phân biệt Kết tập / Thành phần

- **Aggregation** (hình thoi rỗng ◇): "has-a" — thành phần có thể tồn tại độc lập.
Ví dụ: Lớp học chứa Sinh viên (SV vẫn tồn tại khi lớp giải tán).
- **Composition** (hình thoi đặc ◆): "owns-a" — thành phần **không** tồn tại độc lập. Ví dụ: Visitor Center chứa Lobby (Lobby bị phá khi tòa nhà bị phá).

Bài tập 1



- Phân tích xây dựng các lớp:
 - Hàng điện máy <mã hàng, tên hàng, nhà sản xuất, giá, thời gian bảo hành, điện áp, công suất>
 - Hàng sành sứ < mã hàng, tên hàng, nhà sản xuất, giá, loại nguyên liệu>
 - Hàng thực phẩm <mã hàng, tên hàng, nhà sản xuất, giá, ngày sản xuất, ngày hết hạn dùng>
- Vẽ sơ đồ lớp UML biểu diễn các lớp trên
- Viết chương trình tạo mỗi loại một mặt hàng cụ thể. Xuất thông tin về các mặt hàng này

Bài tập 2

- Một phòng ban gồm có các nhân viên là cộng tác viên, nhân viên chính thức, trưởng phòng. Mỗi nhân viên cần quản lý các thông tin chung như:
 - Mã nhân viên, họ tên, năm sinh, giới tính, địa chỉ, hệ số lương (giá trị > 0) và lương tối đa.
 - Cộng tác viên có thêm thông tin về thời hạn hợp đồng (có các giá trị: "3 tháng", "6 tháng", "1 năm") và có khoản phụ cấp lao động được cộng thêm vào thu nhập.
 - Nhân viên chính thức có thêm thông tin về vị trí công việc còn lương thì tính như thông thường.
 - Trưởng phòng có thêm thông tin về ngày bắt đầu quản lý và có khoản phụ cấp quản lý được cộng thêm vào thu nhập.
- Hãy xác định các lớp theo mô tả trên
- Xác định các thuộc tính và phương thức của mỗi lớp, biểu diễn bằng ký pháp UML
- Viết mã nguồn Python cho các lớp trên

Bài tập 3

Một đơn vị sản xuất gồm có các cán bộ là công nhân, kỹ sư, nhân viên. Mỗi cán bộ cần quản lý các dữ liệu: Họ tên, tuổi, giới tính (nam, nữ, khác), địa chỉ.

- Cấp công nhân sẽ có thêm các thuộc tính riêng: Bậc (1 đến 10).
- Cấp kỹ sư có thuộc tính riêng: Ngành đào tạo.
- Các nhân viên có thuộc tính riêng: công việc.

Yêu cầu 1: Xây dựng các lớp CongNhan, KySu, NhanVien kế thừa từ lớp CanBo.

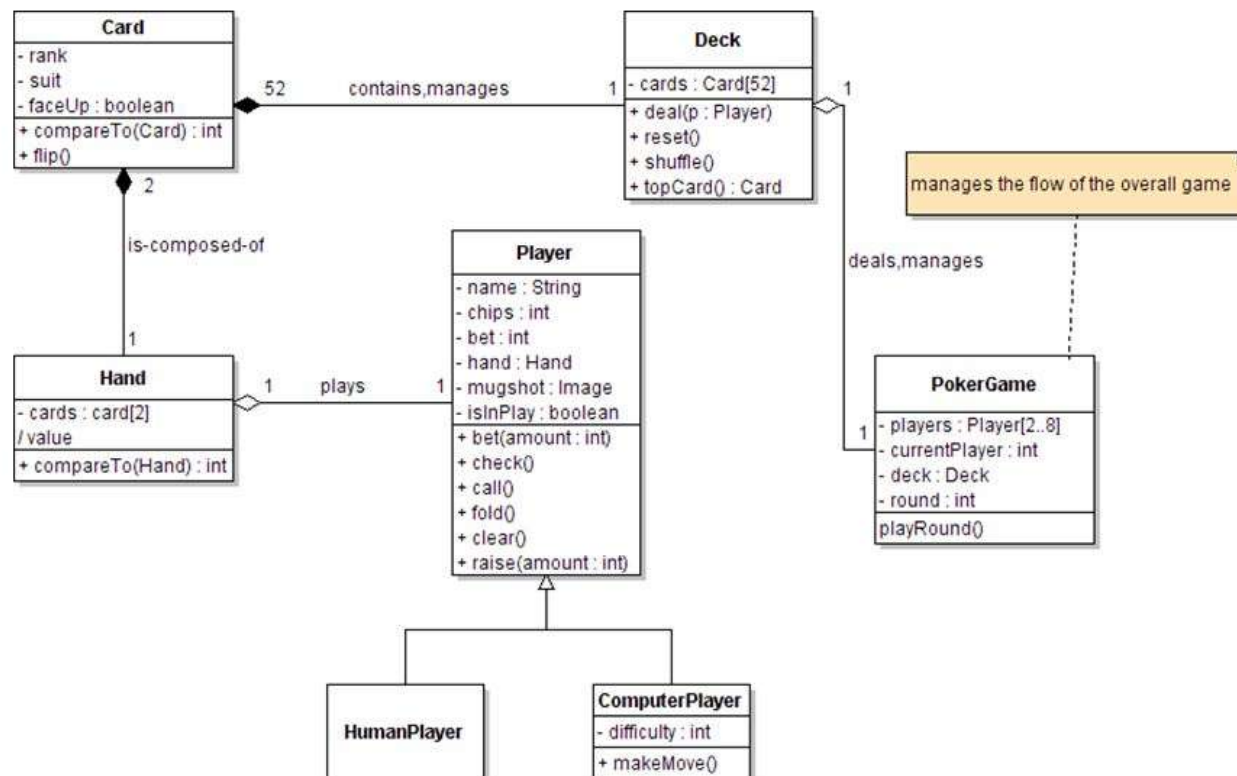
Yêu cầu 2: Xây dựng lớp QLCB(quản lý cán bộ) cài đặt các phương thức thực hiện các chức năng sau:

- Thêm mới cán bộ.
- Tìm kiếm theo họ tên.
- Hiển thị thông tin về danh sách các cán bộ.
- Thoát khỏi chương trình.

Bài tập nâng cao (team work)

- Viết mã nguồn triển khai **game bài Poker** dựa vào sơ đồ lớp dưới đây.

Poker class diagram



Một số ví dụ minh họa bổ sung (1)

Thứ tự khởi tạo trong kế thừa đa cấp: thấy rõ thứ tự constructor khi có kế thừa nhiều cấp (Multilevel):

```
class DongVat:
    def __init__(self, ten):
        self.ten = ten
        print(f"1. DongVat.__init__({ten})")

class ThuNuoi(DongVat):
    def __init__(self, ten, chu_nhan):
        super().__init__(ten)      # Gọi DongVat.__init__ TRƯỚC
        self.chu_nhan = chu_nhan
        print(f"2. ThuNuoi.__init__(chủ={chu_nhan})")

class Cho(ThuNuoi):
    def __init__(self, ten, chu_nhan, giống):
        super().__init__(ten, chu_nhan)  # Gọi ThuNuoi.__init__ TRƯỚC
        self.giống = giống
        print(f"3. Cho.__init__(giống={giống})")

# Tạo đối tượng
cho = Cho("Buddy", "An", "Golden Retriever")

# Output:
# 1. DongVat.__init__(Buddy)
# 2. ThuNuoi.__init__(chủ=An)
# 3. Cho.__init__(giống=Golden Retriever)

# Kiểm tra chuỗi kế thừa
print(Cho.__mro__)
# [Cho, ThuNuoi, DongVat, object]
```

Một số ví dụ minh họa bổ sung (2)

Method Overriding (ghi đè phương thức): Minh họa lớp con có thể thay đổi hành vi lớp cha:

```
class HìnhHoc:
    def __init__(self, ten):
        self._ten = ten

    def dien_tich(self):
        return 0 # Mặc định, sẽ bị ghi đè

    def __str__(self):
        return f"{self._ten}: S = {self.dien_tich()}"

class HìnhTron(HìnhHoc):
    def __init__(self, ban_kinh):
        super().__init__("Hình tròn")
        self._r = ban_kinh

    def dien_tich(self): # GHI ĐÈ (Override)
        return 3.14159 * self._r ** 2
```

```
class HìnhChuNhat(HìnhHoc):
    def __init__(self, dai, rong):
        super().__init__("Hình chữ nhật")
        self._dai = dai
        self._rong = rong

    def dien_tich(self): # GHI ĐÈ (Override)
        return self._dai * self._rong

class HìnhVuong(HìnhChuNhat): # Kế thừa từ HCN
    def __init__(self, canh):
        super().__init__(canh, canh) # HV là HCN có dài = rộng
        self._ten = "Hình vuông"

# Demo
shapes = [HìnhTron(5), HìnhChuNhat(4, 6), HìnhVuong(3)]
for s in shapes:
    print(s)
# Hình tròn: S = 78.53975
# Hình chữ nhật: S = 24
# Hình vuông: S = 9
```

Một số ví dụ minh họa bổ sung (3)

Name Mangling (giải thích chi tiết): hiểu rõ cơ chế "private" trong Python:

```
class A:
    def __init__(self):
        self.__secret = "Bí mật của A"
        self._protected = "Protected của A"
        self.public = "Public của A"

class B(A):
    def __init__(self):
        super().__init__()
        self.__secret = "Bí mật của B"  # Đây là _B__secret, KHÔNG ghi đè _A__secret

b = B()

# Xem tất cả thuộc tính thực sự của đối tượng
print([attr for attr in dir(b) if 'secret' in attr])
# ['_A__secret', '_B__secret'] ← Cả 2 đều tồn tại!

print(b._A__secret)    # "Bí mật của A" ← vẫn truy cập được
print(b._B__secret)    # "Bí mật của B"
print(b._protected)    # "Protected của A"
print(b.public)        # "Public của A"
```


Tổng kết (1)

KẾ THỪA (INHERITANCE) TRONG OOP



Kế thừa là gì?

Xây dựng lớp mới từ lớp đã có
Quan hệ **is-a** (là một loại của)



Cú pháp

Python: **class Con(Cha):**
Java: **class Con extends Cha**



Nguyên lý kế thừa

public / protected → kế thừa được
__private → name mangling (bị đổi tên)



Khởi tạo & super()

super().__init__(args)
Lớp cha khởi tạo trước → lớp con sau



Các loại kế thừa

Single → Multiple → Multilevel
Hierarchical → Hybrid



Composition vs Aggregation

◆ **Composition**: thành phần phụ thuộc
◇ **Aggregation**: thành phần độc lập

Tổng kết (2)

SO SÁNH CÁC QUAN HỆ

Tiêu chí	Kế thừa (Inheritance)	Thành phần (Composition)
Quan hệ	is-a (là một loại)	has-a / owns-a (sở hữu)
Ví dụ	SinhVien là Nguoi	VisitorCenter có Lobby
Tái sử dụng qua	Lớp (class)	Đối tượng (object)
Vòng đời	Độc lập	Phụ thuộc (hủy cha → hủy con)
UML	▷ Mũi tên tam giác rỗng	◆ Hình thoi đặc

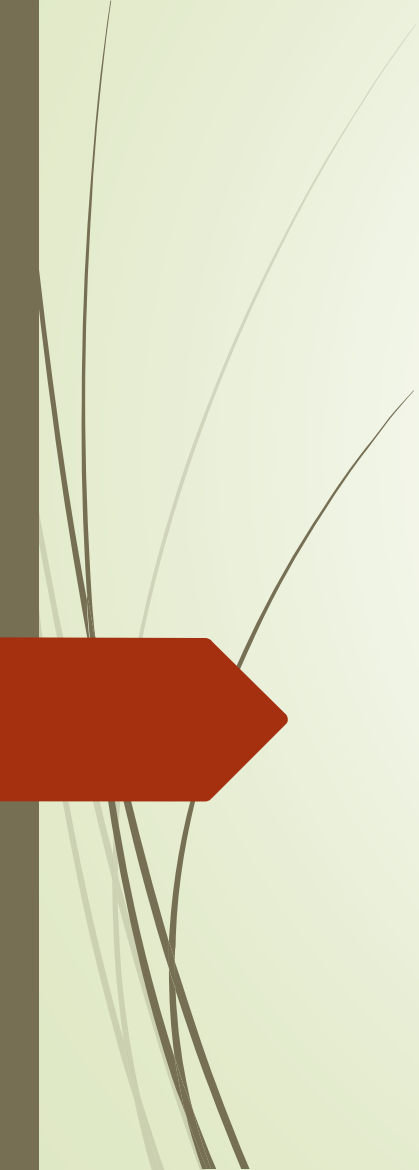
LƯU Ý QUAN TRỌNG TRONG PYTHON

1 Access modifier chỉ là **quy ước**, không phải cơ chế cứng như Java/C++

2 `__private` bị name mangling → `_TenLop_attr`

3 `super().__init__()` phải gọi ở dòng đầu tiên của constructor lớp con

4 Python hỗ trợ **đa kế thừa** — dùng MRO - Method Resolution Order (C3 linearization) giải quyết xung đột

A decorative graphic on the left side of the slide featuring several thin, curved lines representing grass or reeds. At the bottom left, there is a solid red arrow pointing to the right.

Thank you!

Any questions?